

11. Data Model and Schema Specification

Canonical entities, storage design, lineage, and schema rules for QuantPipeline

Project: QuantPipeline

Document status: Working baseline for product and engineering delivery

Version: v1.0

Date: 2026-04-23

Audience: Product owner, architecture, development, QA, and future implementation agents

Purpose: This document is intended to be implementable. It defines contracts, boundaries, and decisions that should survive future feature growth without breaking the core recommendation pipeline.

Document Control

Field	Value
Document code	11
Title	Data Model and Schema Specification
Primary input set	Documents 01-09, A1 technology recommendation, and baseline truth table
Primary architecture anchor	Data & Feature Layer -> Signals -> Selection -> Allocation -> Timing -> Risk Overlay -> Recommendation Object -> Backtest / Paper / Replay / Ops
Assumed release posture	Private single-user first release with future-safe modular boundaries
Key dependencies	Documents 10 and 12, 08 PRD, 09 Functional Requirements
Quality rule	No silent degradation, explicit trust semantics, and replayable output

1. Purpose and Modeling Scope

This document defines the canonical data model for QuantPipeline. It specifies the main entities, tables, relationships, identifiers, temporal rules, lineage requirements, and storage conventions required to support one coherent recommendation pipeline across operational usage, replay, backtesting, paper shadowing, and admin oversight.

The central design rule is that decisions must be reconstructable. It is not enough to store the final recommendation. The schema must preserve what was known, which features existed, which policy versions were active, what each stage decided, which guardrails modified the output, and what was finally published or suppressed.

2. Data Modeling Principles

- Stable identifiers over mutable labels.
- Append-only history for decision outputs and publication records.
- Explicit timestamps for event time, processing time, and publication time.
- Freshness and confidence as stored fields, not implied behavior.
- Separation of raw source preservation from curated analytical state.
- Separation of canonical domain entities from denormalized read models.
- Schema support for both production recommendations and historical validation artifacts.
- Version every policy, model, feature recipe, and manual override that can affect output.

3. Canonical Data Domains

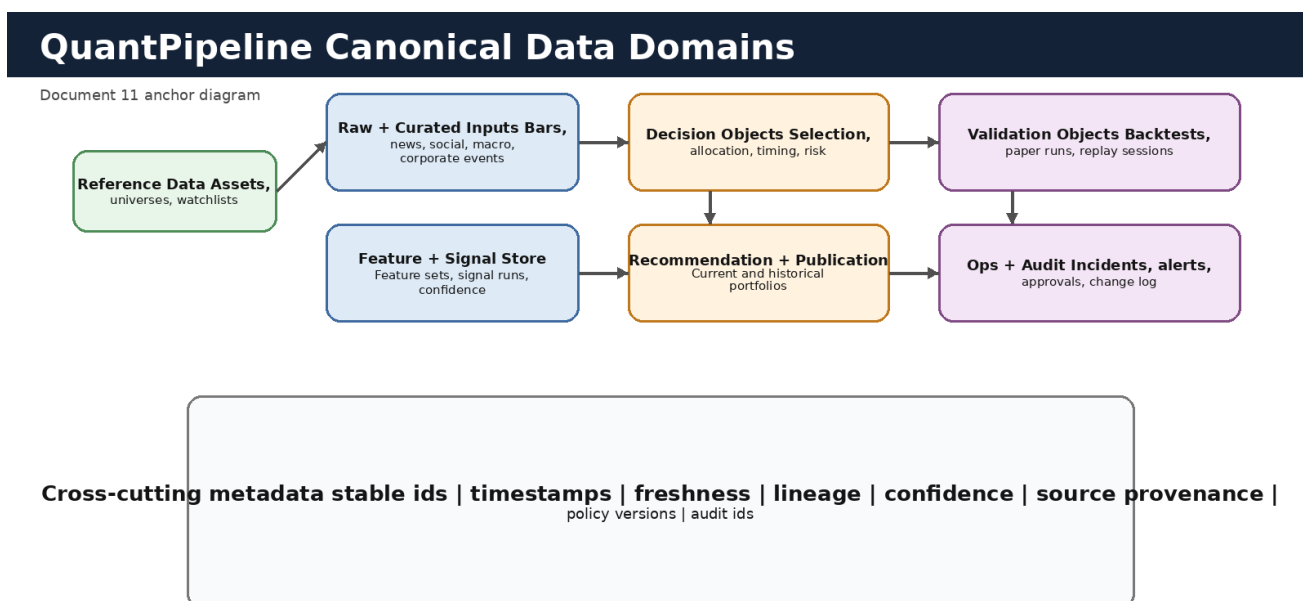


Figure 11-1. Canonical data domains and their metadata spine.

The schema is organized into seven domains: reference data, raw and curated inputs, feature and signal store, decision-stage objects, recommendation and publication history, validation artifacts, and ops/audit records. A shared metadata spine provides stable ids, time semantics, lineage, confidence, and policy versioning across all domains.

4. Identifier and Time Semantics

Every persisted object that can appear in a user-facing flow should have a stable identifier. Internal row ids may exist for convenience, but they must not be the sole reference mechanism for recommendation runs, replay ids, experiment ids, or incident ids.

QuantPipeline requires three families of time fields. Event time captures when the market or text event actually occurred. Processing time captures when QuantPipeline ingested or derived the object. Effective time captures the time window for which a feature set, policy output, or recommendation is considered valid.

Where historical correction is possible, preserve both original source timestamps and normalized timestamps. Do not overwrite raw timestamps during normalization.

Field family	Example	Why it exists
Stable ids	asset_id, recommendation_id, experiment_id	Cross-table joins, API references, replay
Event time	bar_close_ts, article_published_ts	Temporal alignment and causality
Processing time	ingested_at,	Operational freshness and

Field family	Example	Why it exists
	feature_computed_at	debugging
Effective window	valid_from, valid_to	Publication and scenario comparisons
Version refs	policy_version_id, model_version_id	Reproducibility and governance

5. Reference Data Domain

Reference data defines what can be analyzed and how instruments are grouped. It includes instruments, universes, watchlists, classification mappings, benchmark definitions, and user-managed scenario sets.

Table	Purpose	Key columns
assets	Canonical tradable instruments	asset_id, ticker, asset_type, currency, exchange, is_active
asset_aliases	Ticker and vendor symbol resolution	alias_id, asset_id, provider, alias_value
asset_classifications	Sector, industry, theme, region	classification_id, asset_id, taxonomy, value
universes	Named investment universes	universe_id, name, scope_type, status
universe_membership	Membership history	universe_id, asset_id, effective_from, effective_to
watchlists	User-defined collections	watchlist_id, owner_id, name, notes
watchlist_membership	Watchlist history	watchlist_id, asset_id, added_at, removed_at
benchmarks	Benchmark definitions	benchmark_id, name, methodology, currency

6. Raw and Curated Input Domain

Raw inputs should be preserved in immutable form where practical. Curated inputs should normalize schema, resolve timestamps, assign source quality fields, and attach lineage links back to the raw payload archive. This allows later reprocessing without pretending that the original feed was cleaner than it was.

Table or store	Purpose	Key fields
source_feeds	Catalog of upstream providers and collectors	source_id, source_type, owner, reliability_tier
raw_payload_manifest	Object-store pointer per raw batch	payload_id, source_id, collected_at, uri, checksum
market_bars	Curated price/volume bars	asset_id, interval, event_ts, open, high, low, close, volume, source_id
news_events	Normalized articles and metadata	news_id, source_id, published_ts, title, language,

Table or store	Purpose	Key fields
		uri
news_asset_links	Links between articles and assets	news_id, asset_id, link_strength, method
social_events	Posts and aggregates	social_id, source_id, published_ts, channel, author_hash, content_ref
macro_events	Macro releases and indicators	macro_id, series_key, event_ts, value, surprise, revision_flag
portfolio_snapshots	Observed actual portfolio state or user reference portfolio	portfolio_snapshot_id, observed_at, holdings_json_ref

7. Feature Registry Domain

Feature generation should be treated as a governed data product. A feature set is not merely a temporary array in memory. It is a named, versioned, time-bounded object with completeness, lineage, and schema definition so that downstream signals and validation can refer to it reliably.

Table	Purpose	Key columns
feature_recipes	Named feature definitions	feature_recipe_id, name, version, owner, description
feature_sets	Materialized feature collection for a run window	feature_set_id, universe_id, effective_ts, status, completeness_score
feature_values	Atomic feature values by asset	feature_set_id, asset_id, feature_key, value_num, value_text, freshness_age_sec
feature_lineage	Links from feature sets to source batches	feature_set_id, payload_id, transformation_step
feature_anomalies	Missing, outlier, or alignment issues	anomaly_id, feature_set_id, severity, rule_key, message

8. Signal and Engine Output Domain

Signal outputs should preserve engine individuality without breaking downstream comparability. Each engine may have specialized fields, but every signal run must also conform to a common minimum contract.

Table	Purpose	Common contract fields
signal_runs	Run header per engine and context	signal_run_id, engine_id, feature_set_id, run_ts, confidence, status
signal_asset_outputs	Per-asset score or classification	signal_run_id, asset_id, score, direction, confidence, warning_flags
signal_rationales	Short explanation fragments	signal_run_id, asset_id,

Table	Purpose	Common contract fields
		rationale_summary, evidence_refs
signal_artifacts	Pointers to model or explainability artifacts	signal_run_id, artifact_type, uri, checksum
engine_versions	Versioned engine registry	engine_id, version, engine_family, promoted_at, status

Minimum contract: Every signal run should be linkable to the feature_set_id, engine_version, effective time, confidence, warnings, and rationale fragments that explain the result.

9. Decision Pipeline Domain

The decision pipeline is the schema heart of QuantPipeline. Selection, allocation, timing, and risk overlay each get their own records and their own change explanations. This allows the system to answer what changed, where it changed, and why the final portfolio differs from the pre-risk proposal.

Table	Purpose	Key columns
selection_runs	Universe filtering and ranking result	selection_run_id, universe_id, policy_version_id, candidate_count, shortlist_count
selection_members	Ranked shortlist members	selection_run_id, asset_id, rank, inclusion_reason, exclusion_reason
allocation_runs	Pre-risk target allocation	allocation_run_id, selection_run_id, allocator_version_id, gross_exposure
allocation_weights	Per-asset proposed weights	allocation_run_id, asset_id, target_weight, capital_bucket
timing_runs	Timing controller output	timing_run_id, allocation_run_id, timing_policy_version_id, posture, aggressiveness_factor
timing_adjustments	Per-asset or portfolio timing changes	timing_run_id, asset_id, delta_weight, rule_trigger
risk_overlay_runs	Portfolio-level risk pass	risk_run_id, timing_run_id, risk_policy_version_id, status
risk_adjustments	Exposure trims, caps, suppressions	risk_run_id, asset_id, before_weight, after_weight, reason

10. Recommendation and Publication Domain

Recommendation records are not only stage outputs. They are a user-facing contract. A recommendation record should capture the published portfolio, its explanation summary, status, freshness, confidence decomposition, and links to every contributing stage object.

Table	Purpose	Key columns
recommendation_runs	Canonical final proposal per context	recommendation_id, risk_run_id, current_status, overall_confidence, horizon
recommendation_weights	Published or proposed target weights	recommendation_id, asset_id, target_weight, rationale_snippet
recommendation_diffs	Change against previous recommendation	recommendation_id, previous_recommendation_id, material_change_flag, summary
publication_records	Published, suppressed, or staged states	publication_id, recommendation_id, publication_state, published_at, approver_id
publication_reasons	Reason codes for non-normal publication	publication_id, reason_type, reason_code, message
current_recommendation_view	Read model for fast dashboard access	context_key, recommendation_id, freshness_age_sec, health_state

11. Validation, Backtest, Paper, and Replay Domain

Historical evaluation objects must be isolated from live publication while still using the same canonical contract shapes. The schema should allow experiments to reference frozen inputs, policy packages, and result artifacts without polluting production recommendation history.

Table	Purpose	Key columns
experiment_runs	Validation run header	experiment_id, methodology_version, policy_bundle_id, frozen_snapshot_id, status
backtest_results	High-level metrics per experiment	experiment_id, benchmark_id, cagr, drawdown, turnover, sharpe
backtest_period_results	Window-level metrics	experiment_id, period_key, return, benchmark_return, regime_tag
paper_portfolios	Named paper portfolios	paper_portfolio_id, policy_bundle_id, inception_ts, status
paper_rebalances	Historical paper rebalance actions	paper_rebalance_id, paper_portfolio_id,

Table	Purpose	Key columns
		recommendation_id, executed_ts
replay_sessions	Replay request or cached reconstruction	replay_id, recommendation_id, generated_at, narrative_uri
replay_stage_views	Stage-specific replay data	replay_id, stage_name, object_ref, delta_summary

12. Admin, Ops, and Governance Domain

Admin and ops data must explain system health in business terms rather than raw log fragments. This domain stores incidents, alerts, approvals, suppressions, manual notes, policy promotions, and operator actions.

Table	Purpose	Key columns
alerts	Current and historical alert instances	alert_id, alert_type, severity, state, raised_at, linked_object_ref
incidents	Operator-managed incident records	incident_id, severity, status, owner_id, opened_at, resolved_at
audit_events	Immutable audit trail	audit_event_id, actor_id, action_type, object_ref, occurred_at, payload_ref
approvals	Approval and rejection records	approval_id, object_ref, workflow_type, decision, actor_id, decided_at
policy_versions	Selection/allocation/timing/risk bundle registry	policy_version_id, bundle_type, semantic_version, promoted_at
manual_overrides	Explicit operator overrides	override_id, object_ref, override_type, reason, expires_at

13. Read Models and API-Optimized Views

Not every user-facing response should be assembled from normalized tables at request time. The schema should include read models or materialized views optimized for the overview, decision breakdown, admin command center, and replay summary screens.

Read models should be rebuildable from canonical state. They are convenience layers, not the source of truth.

Read model	Purpose	Built from
overview_snapshot_view	Fast current recommendation summary	recommendation_runs, publication_records, current

Read model	Purpose	Built from
		freshness metrics
decision_breakdown_view	Selection/allocation/timing/ risk explainer	decision-stage tables and rationale fragments
engine_comparison_view	Cross-engine matrix	signal_runs and signal_asset_outputs
ops_command_center_view	Admin home screen	alerts, incidents, queue and freshness aggregates
paper_monitor_view	Paper status and drift summary	paper_rebalances, portfolio snapshots, recommendation diffs

14. Retention, Partitioning, and Index Strategy

Retention should reflect product value, legal sensitivity, and operational cost. Recommendation history, stage outputs, approvals, and audit events should be kept long enough to support meaningful replay and trust analysis. High-volume raw payloads and feature values may be tiered between hot and cold storage.

Partition the heaviest time-series and event tables by date and, where useful, by interval or source. Index for the most common joins: asset plus time, recommendation plus stage reference, experiment plus benchmark, and alert plus state.

Data family	Suggested retention posture	Notes
Recommendation and stage history	Long-term	Required for replay and trust
Raw payload manifests	Long-term manifest, cold-store payloads	Keep checksums and URIs even if blobs are tiered
Feature values	Tiered by age	Hot recent windows, cold archive for reproducibility
Logs and metrics	Policy-driven operational retention	Separate from domain retention
Paper and validation artifacts	Long-term for promoted methods; tiered otherwise	Preserve evidence for policy changes

15. Migration and Schema Evolution Rules

- Use forward-only, version-controlled migrations.
- Never change a historical meaning of a status or risk reason without explicit migration and mapping.
- Prefer additive schema changes over destructive ones when historical replay would be impacted.
- Treat changes to ids, timestamps, or publication-state semantics as high-risk migrations requiring rehearsal.
- Preserve compatibility windows between Document 11 schemas and Document 12 API contracts.

- When introducing a new stage or engine family, add canonical metadata fields before adding specialized payloads.

Appendix A - Canonical Recommendation JSON Skeleton

```
{ "recommendation_id": "...", "as_of": "...", "context": { "universe_id": "...", "horizon":  
"medium_term"}, "stage_refs": { "selection_run_id": "...", "allocation_run_id": "...", "timing_run_id":  
"...", "risk_run_id": "..."}, "portfolio": [{ "asset_id": "...", "target_weight": 0.08}], "confidence":  
{ "overall": 0.72, "data": 0.81, "model": 0.69, "operational": 0.93}, "status": { "publication_state":  
"published", "health_state": "healthy", "warnings": [], "audit_ref": "..." }
```

Appendix B - Table Creation Priority

1. Reference data, raw manifests, market_bars, and news_events.
2. Feature registry and signal runs.
3. Selection, allocation, timing, and risk overlay tables.
4. Recommendation and publication records plus current read models.
5. Validation, paper, replay, alerts, incidents, approvals, and audit events.